

# Implementation report: $\alpha$ -attractor inflaton models in COSMOLATTICE

Covers arXiv:2510.18656 (Ellis, Garcia, Olive, Verner)

October 2, 2026

## Abstract

This report documents the COSMOLATTICE implementation of every inflationary model found viable in arXiv:2510.18656: the generalized and deformed  $\alpha$ -attractor E-models and T-models. It lists the new files, explains how the model headers and the initial-condition helper were built, and justifies every parameter choice in the benchmark parameter files. All families are covered by two model headers, `attractorE.h` and `attractorT.h`, plus 14 parameter files. It also derives the expressions used to redshift the lattice GW spectra and wavenumbers to today's frequency and  $h^2\Omega_{\text{GW}}$  (Sec. 8). The physics background (data constraints, derivative formulas, analytic approximations) is in the companion note `notes/inflation_models.tex`.

## Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Summary</b>                                   | <b>1</b>  |
| <b>2</b> | <b>New and modified files</b>                    | <b>2</b>  |
| 2.1      | New files . . . . .                              | 2         |
| 2.2      | Modified files . . . . .                         | 2         |
| <b>3</b> | <b>Models implemented</b>                        | <b>2</b>  |
| <b>4</b> | <b>How the model headers were built</b>          | <b>3</b>  |
| 4.1      | Run-time parameters . . . . .                    | 3         |
| 4.2      | Program variables . . . . .                      | 3         |
| 4.3      | Numerically safe forms . . . . .                 | 4         |
| <b>5</b> | <b>The initial-condition helper</b>              | <b>4</b>  |
| <b>6</b> | <b>Choice of the parameters</b>                  | <b>5</b>  |
| 6.1      | Model parameters $(\alpha, k, \kappa)$ . . . . . | 5         |
| 6.2      | Number of $e$ -folds $N_*$ . . . . .             | 5         |
| 6.3      | Lattice and run settings . . . . .               | 6         |
| 6.4      | Benchmark files . . . . .                        | 6         |
| <b>7</b> | <b>Validation</b>                                | <b>6</b>  |
| <b>8</b> | <b>Redshifting the GW spectrum to today</b>      | <b>7</b>  |
| 8.1      | Times, notation and assumptions . . . . .        | 7         |
| 8.2      | Frequency today . . . . .                        | 8         |
| 8.3      | Amplitude today . . . . .                        | 9         |
| 8.4      | Implementation . . . . .                         | 9         |
| 8.5      | Results for the two runs . . . . .               | 10        |
| <b>9</b> | <b>How to build and run</b>                      | <b>10</b> |

## 1 Summary

- **Two model headers** cover the four viable families. A deformed model with  $\kappa = 1$  is exactly the corresponding generalized model, and  $k = 2$ ,  $\alpha = 1$ ,  $\kappa = 1$  of the E-model is Starobinsky.  $\alpha$ ,  $k$ ,  $\kappa$ ,  $\lambda$  and the daughter coupling  $g$  are read at run time, so no recompilation is needed to scan them.
- **One Python helper** (`notes/attractor_ics.py`) solves the homogeneous background. Given  $(\alpha, k, \kappa, N_*)$  it returns the CMB normalization  $\lambda$  and the inflaton amplitude and velocity at the end of inflation.
- **14 benchmark parameter files** reproduce the points quoted in the paper.
- Both models compile against COSMOLATTICE 2.0 and ran without errors in short tests (Sec. 7).

## 2 New and modified files

### 2.1 New files

| File                                                                           | Content                                                                                          |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>models/attractorE.h</code>                                               | Generalized + deformed E-model ( $\alpha$ -Starobinsky), inflaton + daughter field.              |
| <code>models/attractorT.h</code>                                               | Generalized + deformed T-model, inflaton + daughter field.                                       |
| <code>models/parameter-files/attractorE_*.in</code><br>(8 files)               | E-model benchmarks (Table 1).                                                                    |
| <code>models/parameter-files/attractorT_*.in</code><br>(6 files)               | T-model benchmarks (Table 1).                                                                    |
| <code>notes/attractor_ics.py</code>                                            | Background solver: $\lambda$ , $\varphi_i$ , $\dot{\varphi}_i$ from $(\alpha, k, \kappa, N_*)$ . |
| <code>notes/inflation_models.tex</code> (+ PDF)                                | Physics note: which models, why, potentials, derivatives, benchmark table.                       |
| <code>notes/implementation_report.tex</code> (+ PDF)                           | This report.                                                                                     |
| <code>runs/E_starobinsky_gw/</code> , <code>runs/T_k2_a1_gw/</code>            | Preheating runs with GWs ( <code>run.in</code> + output).                                        |
| <code>runs/attractor_runs.ipynb</code> ,<br><code>runs/make_notebook.py</code> | Analysis notebook and its generator; Sec. 5 redshifts the GW spectra to today (Sec. 8).          |

Two build directories, `build_attractorE/` and `build_attractorT/`, were also created. They are ignored by git.

### 2.2 Modified files

No tracked source file of COSMOLATTICE was modified: the new models are self-contained headers picked up by the existing CMake option `-DMODEL=<name>`.

Before this work the repository was updated to the latest upstream version (COSMOLATTICE 2.0, commit `acc8278d`). Two local changes to tracked files (deletion of `src/models/parameter-files/lphi4.in`, edit of `lphi4U1.in`) were reverted to the upstream versions. They are saved in git `stash` entry “local changes before pulling v2.0”. Your untracked files (`src/models/lphi4_only.h`, `m2phi2.h`, their `.in` files, `papers/`) were not touched. In v2.0 models live in `models/`, not `src/models/`.

### 3 Models implemented

With  $x \equiv \varphi/M_P$ ,  $b \equiv \sqrt{2/(3\alpha)}$ ,  $c \equiv b/2 = 1/\sqrt{6\alpha}$  and  $V_0 = \frac{3}{4}\lambda M_P^4$ :

$$\text{attractorE: } V = V_0 [\kappa(1 - \cosh bx) + \sinh bx]^k, \quad (1)$$

$$\text{attractorT: } V = V_0 \left[ \frac{1}{2}(1 + \kappa + (1 - \kappa) \cosh bx) \right]^2 \tanh^k(cx). \quad (2)$$

These are the deformed models of the paper (its Sec. V.B, “Deformed No-Scale Attractors”). They reduce to

- $\kappa = 1$ : generalized E-model  $V_0(1 - e^{-bx})^k$  and generalized T-model  $V_0 \tanh^k(cx)$  (paper Sec. V.A);
- $\kappa = 1$ ,  $k = 2$ : standard E- and T-models; additionally  $\alpha = 1$ : Starobinsky.

We checked analytically that the paper’s superpotential construction (its Appendix A) gives the bracket in Eq. (1) raised to the power  $k$ . Near the minimum both potentials behave as  $\varphi^k$  for any  $\kappa$ .

A second scalar  $\chi$  (field index 1) is coupled through  $\frac{1}{2}g^2\varphi^2\chi^2$ , as in the existing `lphi4.h` and `tanh2.h`. The default is  $g = 0$ : then  $\chi$  is a free massless spectator and the run studies the inflaton’s self-resonance only.

### 4 How the model headers were built

The headers follow the structure of the existing `models/tanh2.h` and `models/lphi4.h`: a `ModelPars` struct (2 scalars, 2 potential terms, `double`), a constructor that reads parameters and sets the program-variable rescaling, and the functions `potentialTerms`, `potDeriv`, `potDeriv2` for each field.

#### 4.1 Run-time parameters

| Key                             | Default  | Symbol                          | Meaning                                                                                                                           |
|---------------------------------|----------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>lambda</code>             | required | $\lambda$                       | CMB normalization, $V_0 = \frac{3}{4}\lambda M_P^4$                                                                               |
| <code>alphaAtt</code>           | 1        | $\alpha$                        | attractor parameter. Not called <code>alpha</code> , which COSMOLATTICE already uses for the time-rescaling exponent              |
| <code>k</code>                  | 2        | $k$                             | power at the minimum; must be an even integer $\geq 2$ (checked, else the run stops with <code>RunParametersInconsistent</code> ) |
| <code>kappa</code>              | 1        | $\kappa$                        | plateau deformation                                                                                                               |
| <code>g</code>                  | 0        | $g$                             | inflaton–daughter coupling                                                                                                        |
| <code>initial_amplitudes</code> | required | $\varphi_i, \chi_i$             | GeV                                                                                                                               |
| <code>initial_momenta</code>    | 0 0      | $\dot{\varphi}_i, \dot{\chi}_i$ | GeV <sup>2</sup>                                                                                                                  |

Since  $k$  is a run-time number, powers are written as `pow(expr, k)` on field expressions. TempLat supports this, and an integer-valued floating exponent is well defined for negative bases, which occur for  $x < 0$ .

#### 4.2 Program variables

COSMOLATTICE evolves  $\tilde{\varphi} = \varphi/f_*$  in the time variable  $d\tilde{\eta} = a^{-\alpha_t}\omega_* dt$ . The program potential is  $\tilde{V} = V/(f_*^2\omega_*^2)$ . We chose:

$$f_* = M_P, \quad \omega_* = \sqrt{\lambda_k} M_P x_i^{(k-2)/2}, \quad \alpha_t = \frac{3(k-2)}{k+2}, \quad (3)$$

where  $x_i = |\varphi_i|/M_P$  and  $\lambda_k$  is defined by the small-field limit  $V \simeq (\lambda_k/k)M_P^{4-k}\varphi^k$ :

$$\lambda_k^{(E)} = k \frac{3}{4} \lambda b^k, \quad \lambda_k^{(T)} = k \frac{3}{4} \lambda c^k. \quad (4)$$

### Why these choices.

- $f_* = M_P$ : the program field is  $x = \varphi/M_P$ , so the potential is written directly in terms of the functions of Sec. 3, with  $b, c$  unchanged. The plateau and the minimum are both  $\mathcal{O}(1)$  in these units.
- $\omega_*$  is the oscillation frequency of a condensate of amplitude  $\varphi_i$  in the  $\varphi^k$  potential. For  $k = 2$  it is the inflaton mass:  $\omega_* = m_\varphi = \sqrt{\lambda/\alpha} M_P$  (E),  $\frac{1}{2}\sqrt{\lambda/\alpha} M_P$  (T). For  $k = 4$  it reduces to the `lphi4.h` choice  $\omega_* = \sqrt{\lambda_4} \varphi_i$ . One oscillation therefore takes  $\Delta\tilde{\eta} \sim 2\pi$  at the start.
- $\alpha_t$ : in a  $\varphi^k$  potential the amplitude decays as  $a^{-6/(k+2)}$ , so the oscillation frequency decays as  $a^{-3(k-2)/(k+2)}$ . With  $\alpha_t = 3(k-2)/(k+2)$  the frequency stays constant in program time. This gives  $\alpha_t = 0$  for  $k = 2$  (as in `tanh2.h`) and  $\alpha_t = 1$  for  $k = 4$  (as in `lphi4.h`);  $\alpha_t = 3/2$  for  $k = 6$ ,  $\alpha_t = 2$  for  $k = 10$ .

With these choices, the code computes two constants once in the constructor:

$$\text{pref} = \frac{3}{4} \lambda \frac{M_P^2}{\omega_*^2}, \quad \text{qInt} = \frac{g^2 M_P^2}{\omega_*^2}, \quad (5)$$

so that  $\tilde{V} = \text{pref} \cdot F(x) + \frac{1}{2} \text{qInt} x^2 \tilde{\chi}^2$ , with  $F = V/V_0$  the bracket expressions of Eqs. (1)–(2).

### 4.3 Numerically safe forms

Near the minimum  $\kappa(1 - \cosh bx)$  and  $(1 - \kappa)(\cosh bx - 1)$  suffer from cancellation. The code uses the identity  $\cosh y - 1 = 2 \sinh^2(y/2)$ :

$$G \equiv \kappa(1 - \cosh bx) + \sinh bx = \sinh bx - 2\kappa \sinh^2(bx/2), \quad (6)$$

$$D \equiv \frac{1}{2}(1 + \kappa + (1 - \kappa) \cosh bx) = 1 + (1 - \kappa) \sinh^2(bx/2). \quad (7)$$

The derivatives implemented are

$$\text{E: } F' = kG^{k-1}G', \quad F'' = k[(k-1)G^{k-2}G'^2 + G^{k-1}G''], \quad (8)$$

$$G' = b(\cosh bx - \kappa \sinh bx), \quad G'' = b^2(\sinh bx - \kappa \cosh bx), \quad (9)$$

$$\text{T: } F' = 2DD'T + D^2T', \quad F'' = 2(D'^2 + DD'')T + 4DD'T' + D^2T'', \quad (10)$$

with  $D' = \frac{1}{2}(1 - \kappa)b \sinh bx$ ,  $D'' = \frac{1}{2}(1 - \kappa)b^2 \cosh bx$ ,  $T = \tanh^k(cx)$ ,  $T' = kc \tanh^{k-1}(cx) \text{sech}^2(cx)$ ,  $T'' = kc^2 \tanh^{k-2}(cx) \text{sech}^2(cx) [(k-1) \text{sech}^2(cx) - 2 \tanh^2(cx)]$ . All of them were checked against finite differences of the potential. The second derivatives set the initial effective masses through `setInitialPotentialAndMassesFromPotential()`.

## 5 The initial-condition helper

`notes/attractor_ics.py` needs `numpy` and `scipy`. Any Python 3 with these packages works (e.g. the `cosmolattice` conda environment, if present on the machine).

### Usage.

```
python3 notes/attractor_ics.py E --alpha 1 --k 2 --kappa 1 --nstar 50 # from the
repository root
```

It prints `lambda`, `alphaAtt`, `k`, `kappa`, `initial_amplitudes` and `initial_momenta` in the `.in` syntax, plus a comment line with the slow-roll  $n_s$ ,  $r$ ,  $x_*$ ,  $x_{\text{end}}$  and  $H_{\text{end}}$ .

## Method.

1. The background equation is integrated in  $e$ -folds  $N$ , where the trajectory  $x(N)$  does not depend on  $\lambda$ :

$$x'' = -(3 - \varepsilon_H) \left( x' + \frac{F'(x)}{F(x)} \right), \quad \varepsilon_H = \frac{1}{2} x'^2. \quad (11)$$

It starts on the slow-roll attractor,  $x' = -F'/F$ , at a field value  $x_0$  increased until the slow-roll estimate gives at least  $N_* + 15$   $e$ -folds.

2. The end of inflation is the event  $\varepsilon_H = 1$ . This gives  $x_{\text{end}}$  and  $x'_{\text{end}}$ .
3.  $x_*$  is the value  $N_*$   $e$ -folds before the end, read from the dense ODE solution.
4.  $\lambda$  is fixed from  $A_s = V_*/(24\pi^2 \varepsilon_{V_*} M_P^4)$  with  $A_s = 2.1 \times 10^{-9}$ , i.e.  $\lambda = 32\pi^2 A_s \varepsilon_{V_*}/F(x_*)$ .
5. The initial conditions are set at the end of inflation:

$$\varphi_i = x_{\text{end}} M_P, \quad \dot{\varphi}_i = M_P H_{\text{end}} x'_{\text{end}}, \quad H_{\text{end}}^2 = \frac{V(x_{\text{end}})}{M_P^2(3 - \varepsilon_H)}. \quad (12)$$

The ODE uses `rtol` =  $10^{-10}$ . The quoted  $n_s$ ,  $r$  are slow-roll estimates, accurate to  $\sim 10^{-3}$  in  $n_s$ ; the paper's values come from solving the Mukhanov–Sasaki equation.

**Check against the paper.** For Starobinsky with  $N_* = 55$  the helper gives  $\lambda = 1.57 \times 10^{-10}$  (paper:  $1.6 \times 10^{-10}$ ),  $x_* = 5.32$  (paper: 5.35),  $x_{\text{end}} = 0.615$  (paper's analytic estimate: 0.63),  $n_s = 0.964$ ,  $r = 0.0037$  (paper, exact: 0.9649, 0.00355).

## 6 Choice of the parameters

### 6.1 Model parameters $(\alpha, k, \kappa)$

The benchmarks are the parameter points for which the paper quotes predictions or which it identifies as best fits (its Results section, collected in Table 2 of `inflation_models.tex`):

- reference points: Starobinsky ( $k = 2$ ,  $\alpha = 1$ ) and the  $k = 2$  T-model;
- steeper minima, which raise  $N_*$  and  $n_s$  through a stiffer reheating equation of state  $w = (k - 2)/(k + 2)$ :  $k = 4, 6$  (E),  $k = 6, 10$  (T) at  $\alpha = 1$ , and  $\alpha = 5$  for E with  $k = 6, 10$ , where the paper finds overlap with the ACT region;
- deformed plateaus:  $\kappa = 0.9999$  for  $k = 2$ ,  $\kappa = 0.99999$  for  $k = 4, 6$  (see below), and the deformed E-model  $k = 6$ ,  $\alpha = 4$  point.

**The value of  $\kappa$  for  $k = 4, 6$ .** The paper's text says  $\kappa = 0.9999$  for every deformed benchmark. With that value the  $k = 4, 6$  points give a blue tilt ( $n_s = 0.99$ – $1.04$ ), far from the quoted  $n_s \simeq 0.966$ – $0.970$ . With  $\kappa = 0.99999$  the helper reproduces them: E  $k = 4$ :  $n_s = 0.969$ ,  $r = 0.0044$  (paper 0.970, 0.004); T  $k = 4$ :  $n_s = 0.966$ ,  $r = 0.0042$  (paper 0.966, 0.004); T  $k = 6$ :  $n_s = 0.968$  (paper 0.969). The corresponding figure file in the arXiv source is also named `fulltplots.kappa0p99999.pdf`, so we used  $\kappa = 0.99999$  for these three files. The deformed E  $k = 6$ ,  $\alpha = 4$  point does match  $\kappa = 0.9999$  ( $n_s = 0.973$ ,  $r = 0.017$ ; paper 0.974, 0.016), so it keeps that value. The paper's  $r \simeq 0.012$  for deformed T  $k = 6$  is not reproduced for either  $\kappa$  (we get 0.004).

## 6.2 Number of $e$ -folds $N_*$

$N_*$  is an input of the helper and fixes  $\lambda$  and the slow-roll observables. It does not change the lattice initial conditions much:  $x_{\text{end}}$  does not depend on  $N_*$ , while  $\lambda$  and  $\dot{\varphi}_i$  change at the 10% level. Values were taken from the paper for its conservative reheating window  $10^2 \text{ GeV} \lesssim T_{\text{RH}} \lesssim 10^{10} \text{ GeV}$ :

- $k = 2$ :  $N_* = 50$ , the value the paper uses for reference; it corresponds to  $T_{\text{RH}} \simeq 5 \times 10^7 \text{ GeV}$  for Starobinsky. From the paper’s App. B formula  $N_* \simeq 59.55 - \frac{1}{3} \ln N_* + \frac{1}{3} \ln(T_{\text{RH}}/M_P)$ ,  $N_* \simeq 45.7\text{--}51.8$  across the window.
- $k = 4$ :  $N_* = 55.7$  (E), 55.8 (T). For  $w = 1/3$ ,  $N_*$  does not depend on  $T_{\text{RH}}$  (paper, App. B).
- $k = 6$ :  $N_* = 57\text{--}58$ , the range quoted by the paper.
- $k = 10$ :  $N_* = 60$ , middle of the paper’s 59–61.5.

For  $k \geq 6$  the true  $N_*$  depends on the inflaton fragmentation temperature, which is exactly what the lattice runs can measure. These values should be revisited once that is known.

## 6.3 Lattice and run settings

All 14 files share the same settings, which are untuned starting points:

| Key                    | Value              | Reason                                                                                                                                                                                                      |
|------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>expansion</code> | <code>true</code>  | self-consistent FRW expansion; needed for reheating                                                                                                                                                         |
| <code>evolver</code>   | <code>VV2</code>   | second-order velocity Verlet, as in <code>tanh2.in</code>                                                                                                                                                   |
| <code>N</code>         | 64                 | modest grid for exploration; 32 was used for the tests                                                                                                                                                      |
| <code>kIR</code>       | 0.05               | $H_{\text{end}}/\omega_* \simeq 0.24$ for Starobinsky (the measured initial Friedmann value in program units, $0.058 \simeq 0.24^2$ ), so the box contains modes from above to well below the horizon scale |
| <code>dt</code>        | 0.02               | $\sim 300$ steps per initial oscillation period ( $2\pi$ ); much smaller than the lattice spacing $\delta x = 2\pi/(N k_{\text{IR}}) \simeq 2$                                                              |
| <code>kCutOff</code>   | 2                  | initial vacuum fluctuations only below $\tilde{k} = 2$ , inside the lattice range ( $\tilde{k}_{\text{max}} \simeq \sqrt{3} N k_{\text{IR}}/2 \simeq 2.8$ )                                                 |
| <code>tMax</code>      | 200                | $\sim 30$ oscillations; extend for fragmentation studies                                                                                                                                                    |
| <code>tOutput*</code>  | 0.5 / 5 / 20       | frequent / infrequent / rare measurements                                                                                                                                                                   |
| <code>g</code>         | 0                  | self-resonance only; set $g > 0$ to study preheating into $\chi$                                                                                                                                            |
| <code>withGWs</code>   | <code>false</code> | off for the first runs                                                                                                                                                                                      |

Before production runs, check resolution by varying `N`, `kIR` and `dt`, and watch the Friedmann-constraint output.

## 6.4 Benchmark files

## 7 Validation

1. **Derivatives:**  $F'$  and  $F''$  of all four families were compared with finite differences at a generic point ( $k = 6$ ,  $\kappa = 0.93$ ); agreement to  $\sim 10^{-8}$ .
2. **Compilation:** both headers build against COSMOLATTICE 2.0 (TempLat v1.0.2) with the `cosmolattice` conda environment (cmake 4.2.3, FFTW3).
3. **Short runs** ( $N = 32$ ,  $\tilde{t} \leq 20$ ):

| File (attractor...in) | $\alpha$ | $k$ | $\kappa$ | $N_*$ | $\lambda$              | $\varphi_i$ [GeV]     | $\dot{\varphi}_i$ [GeV <sup>2</sup> ] | $(n_s, r)$ slow roll |
|-----------------------|----------|-----|----------|-------|------------------------|-----------------------|---------------------------------------|----------------------|
| E_k2.a1_starobinsky   | 1        | 2   | 1        | 50    | $1.89 \times 10^{-10}$ | $1.50 \times 10^{18}$ | $-2.79 \times 10^{31}$                | 0.9604, 0.0044       |
| E_k4.a1               | 1        | 4   | 1        | 55.7  | $1.60 \times 10^{-10}$ | $2.68 \times 10^{18}$ | $-2.29 \times 10^{31}$                | 0.9639, 0.0038       |
| E_k6.a1               | 1        | 6   | 1        | 57    | $1.56 \times 10^{-10}$ | $3.53 \times 10^{18}$ | $-2.14 \times 10^{31}$                | 0.9645, 0.0037       |
| E_k6.a5               | 5        | 6   | 1        | 58    | $7.23 \times 10^{-10}$ | $5.16 \times 10^{18}$ | $-2.16 \times 10^{31}$                | 0.9653, 0.0155       |
| E_k10.a5              | 5        | 10  | 1        | 60    | $7.09 \times 10^{-10}$ | $7.37 \times 10^{18}$ | $-1.83 \times 10^{31}$                | 0.9660, 0.0152       |
| E_k2.a1_kappa0p9999   | 1        | 2   | 0.9999   | 50    | $2.61 \times 10^{-10}$ | $1.50 \times 10^{18}$ | $-3.28 \times 10^{31}$                | 0.9737, 0.0062       |
| E_k4.a1_kappa0p99999  | 1        | 4   | 0.99999  | 55.7  | $1.86 \times 10^{-10}$ | $2.68 \times 10^{18}$ | $-2.46 \times 10^{31}$                | 0.9694, 0.0044       |
| E_k6.a4_kappa0p9999   | 4        | 6   | 0.9999   | 57    | $7.54 \times 10^{-10}$ | $4.93 \times 10^{18}$ | $-2.51 \times 10^{31}$                | 0.9729, 0.0169       |
| T_k2.a1               | 1        | 2   | 1        | 50    | $2.08 \times 10^{-10}$ | $2.04 \times 10^{18}$ | $-2.44 \times 10^{31}$                | 0.9591, 0.0049       |
| T_k6.a1               | 1        | 6   | 1        | 58    | $1.55 \times 10^{-10}$ | $4.83 \times 10^{18}$ | $-1.92 \times 10^{31}$                | 0.9646, 0.0037       |
| T_k10.a1              | 1        | 10  | 1        | 60    | $1.45 \times 10^{-10}$ | $6.30 \times 10^{18}$ | $-1.84 \times 10^{31}$                | 0.9658, 0.0034       |
| T_k2.a1_kappa0p9999   | 1        | 2   | 0.9999   | 50    | $2.78 \times 10^{-10}$ | $2.04 \times 10^{18}$ | $-2.82 \times 10^{31}$                | 0.9714, 0.0066       |
| T_k4.a1_kappa0p99999  | 1        | 4   | 0.99999  | 55.8  | $1.80 \times 10^{-10}$ | $3.71 \times 10^{18}$ | $-2.11 \times 10^{31}$                | 0.9659, 0.0042       |
| T_k6.a1_kappa0p99999  | 1        | 6   | 0.99999  | 57    | $1.80 \times 10^{-10}$ | $4.83 \times 10^{18}$ | $-2.07 \times 10^{31}$                | 0.9681, 0.0042       |

Table 1: The 14 benchmark parameter files. All numbers come from `notes/attractor_ics.py`; the generating command is recorded in the header of each file.

| Parameter file                | NaN  | Friedmann rel. error at $\tilde{t} = 19.5$ | Behaviour                          |
|-------------------------------|------|--------------------------------------------|------------------------------------|
| attractorE_k2.a1_starobinsky  | none | $2.3 \times 10^{-4}$                       | damped oscillation around $x = 0$  |
| attractorT_k2.a1              | none | $3.8 \times 10^{-4}$                       | damped oscillation                 |
| attractorE_k6.a1              | none | $4.8 \times 10^{-6}$                       | anharmonic, asymmetric oscillation |
| attractorE_k6.a4_kappa0p9999  | none | $1.1 \times 10^{-5}$                       | anharmonic oscillation             |
| attractorT_k6.a1_kappa0p99999 | none | $2.6 \times 10^{-5}$                       | anharmonic, symmetric oscillation  |

The initial program momentum of the Starobinsky run,  $\tilde{\pi} = -0.342$ , agrees with the hand conversion  $\dot{\varphi}_i/(f_*\omega_*)$ .

**Not yet done:** comparison of `attractorT` ( $k = 2, \kappa = 1$ ) with the existing `tanh2` model (mapping  $M = \sqrt{6\alpha} M_P$ ,  $\Lambda^4 = \frac{3}{2}\lambda M_P^4$ ); resolution studies; long runs.

## 8 Redshifting the GW spectrum to today

The lattice gives the GW spectrum at the end of the run,  $t_f$ , as a function of the comoving wavenumber in program units:

$$\Omega_{\text{GW}}^{(f)}(k) \equiv \frac{1}{\rho_f} \left( \frac{d\rho_{\text{GW}}}{d \ln k} \right)_f, \quad (13)$$

where  $\rho_f$  is the total energy density at  $t_f$  (file `spectra_energy_gws.txt`, columns  $\tilde{k}$ ,  $\Omega_{\text{GW}}$ , multiplicity). To compare it with detectors we need the frequency today,  $f$ , of each mode  $k$ , and the spectrum today,  $h^2\Omega_{\text{GW}}(f)$ . This is done in Sec. 5 of `runs/attractor_runs.ipynb` (code in `runs/make_notebook.py`, function `gw_today`). We follow Figueroa & Torrenti, arXiv:1707.04533, Sec. 2.1, Eqs. (2.16)–(2.18); the derivation is repeated here so that the conventions and the treatment of a time-dependent  $w$  are explicit.

### 8.1 Times, notation and assumptions

- $t_i$ : start of the simulation (end of inflation). COSMOLATTICE sets  $a_i = 1$ , so the comoving  $k$  equals the physical wavenumber at  $t_i$ .
- $t_f$ : end of GW production, taken as the time of the spectrum being redshifted (the last output,  $\tilde{t}_f = 290$ , unless stated otherwise).

- $t_{\text{RD}}$ : onset of radiation domination (RD). From then on the universe is a thermal plasma with entropy conserved.
- $t_o$ : today.  $g_t$  and  $g_{s,t}$  are the relativistic and entropic degrees of freedom at time  $t$ .
- Between  $t_i$  and  $t_{\text{RD}}$  the equation of state  $w = p/\rho$  is arbitrary (and time dependent);  $\rho \propto a^{-3(1+w)}$  for constant  $w$ , and in general

$$\frac{d \ln \rho}{d \ln a} = -3(1+w). \quad (14)$$

- After production, GWs are free waves well inside the horizon: their comoving  $k$  is fixed, and their energy redshifts as radiation,  $d\rho_{\text{GW}}/d \ln k \propto a^{-4}$  for each  $k$ .

## 8.2 Frequency today

A comoving mode  $k$  has physical wavenumber  $k/a$ . Its frequency today is

$$f = \frac{1}{2\pi} \frac{k}{a_o} = \frac{a_i}{a_o} \frac{k}{2\pi}, \quad \frac{a_i}{a_o} = \frac{a_i}{a_{\text{RD}}} \frac{a_{\text{RD}}}{a_o}. \quad (15)$$

**From  $t_{\text{RD}}$  to today.** Entropy conservation,  $g_s T^3 a^3 = \text{const}$ , gives  $a_{\text{RD}}/a_o = (g_{s,o}/g_{s,\text{RD}})^{1/3} T_o/T_{\text{RD}}$ . With  $\rho = \frac{\pi^2}{30} g T^4$  for radiation,  $T_o/T_{\text{RD}} = (g_{\text{RD}}/g_o)^{1/4} (\rho_o/\rho_{\text{RD}})^{1/4}$ , where  $\rho_o$  is the radiation energy density today. Hence

$$\frac{a_{\text{RD}}}{a_o} = \left( \frac{g_{s,o}}{g_{s,\text{RD}}} \right)^{1/3} \left( \frac{g_o}{g_{\text{RD}}} \right)^{-1/4} \left( \frac{\rho_o}{\rho_{\text{RD}}} \right)^{1/4}. \quad (16)$$

**From  $t_i$  to  $t_{\text{RD}}$ .** For constant  $w$ ,  $\rho_{\text{RD}} = \rho_i (a_i/a_{\text{RD}})^{3(1+w)}$ , so  $\rho_{\text{RD}}^{-1/4} = \rho_i^{-1/4} (a_i/a_{\text{RD}})^{-3(1+w)/4}$ . Inserting this and Eq. (16) in Eq. (15), the powers of  $a_i/a_{\text{RD}}$  combine as  $1 - \frac{3}{4}(1+w) = \frac{1}{4}(1-3w)$ :

$$\frac{a_i}{a_o} = \epsilon_i^{1/4} \left( \frac{g_{s,o}}{g_{s,\text{RD}}} \right)^{1/3} \left( \frac{g_o}{g_{\text{RD}}} \right)^{-1/4} \left( \frac{\rho_o}{\rho_i} \right)^{1/4}, \quad \epsilon_i \equiv \left( \frac{a_i}{a_{\text{RD}}} \right)^{1-3w}. \quad (17)$$

This is the paper's Eq. (2.16).  $\epsilon_i < 1$  for  $w < 1/3$  (shift to the IR),  $= 1$  for  $w = 1/3$ ,  $> 1$  for a stiff  $w > 1/3$ .

**Time-dependent  $w$ .** Integrating Eq. (14),  $\rho_{\text{RD}}/\rho_i = \exp[-3 \int_{a_i}^{a_{\text{RD}}} (1+w) d \ln a]$ . Then

$$\frac{a_i}{a_{\text{RD}}} \left( \frac{\rho_i}{\rho_{\text{RD}}} \right)^{-1/4} = \exp \left[ - \int d \ln a + \frac{3}{4} \int (1+w) d \ln a \right] = \exp \left[ - \frac{1}{4} \int_{a_i}^{a_{\text{RD}}} (1-3w) d \ln a \right], \quad (18)$$

so Eq. (17) holds with

$$\epsilon_i = \exp \left[ - \int_{a_i}^{a_{\text{RD}}} (1-3w) d \ln a \right]. \quad (19)$$

For constant  $w$  this reduces to  $(a_i/a_{\text{RD}})^{1-3w}$ .

**Result.** The frequency today is (paper's Eq. (2.17), first line)

$$f = \epsilon_i^{1/4} \left( \frac{g_{s,o}}{g_{s,\text{RD}}} \right)^{1/3} \left( \frac{g_o}{g_{\text{RD}}} \right)^{-1/4} \left( \frac{\rho_o}{\rho_i} \right)^{1/4} \frac{k}{2\pi} \quad (20)$$

With the constants of Sec. 8.4 the numerical prefactor is

$$f \simeq 4.6 \times 10^{10} \epsilon_i^{1/4} \frac{k}{\rho_i^{1/4}} \text{ Hz}, \quad (21)$$

which reproduces the standard  $f \simeq 4 \times 10^{10} (k/\rho_i^{1/4}) \text{ Hz}$  of Dufaux et al. (2007) for  $\epsilon_i = 1$ .

**A  $2\pi$  in the paper.** The second line of the paper's Eq. (2.17),  $f \simeq \epsilon_i^{1/4} (k/\rho_i^{1/4}) \times 8 \cdot 10^9 \text{ Hz}$ , does not follow from its first line. With the paper's  $\rho_o \simeq 2 \cdot 10^{-15} \text{ eV}^4$  and the  $g$  factors set to 1, the first line gives  $\rho_o^{1/4}/(2\pi) = 5.1 \times 10^{10} \text{ Hz}$ , which is  $2\pi$  times  $8 \cdot 10^9 \text{ Hz}$ . We use the first line, Eq. (20).

### 8.3 Amplitude today

After  $t_f$ ,  $d\rho_{\text{GW}}/d\ln k \propto a^{-4}$  at fixed  $k$ , so

$$\Omega_{\text{GW}}(k, t_o) = \frac{1}{\rho_c} \left( \frac{d\rho_{\text{GW}}}{d\ln k} \right)_o = \frac{1}{\rho_c} \left( \frac{a_f}{a_o} \right)^4 \rho_f \Omega_{\text{GW}}^{(f)}(k), \quad (22)$$

with  $\rho_c$  the critical density today. Write  $\rho_f a_f^4$  in terms of the radiation density today,  $\rho_o = \rho_c \Omega_{\text{rad}}$ :

$$\frac{\rho_f a_f^4}{\rho_o a_o^4} = \underbrace{\frac{\rho_f a_f^4}{\rho_{\text{RD}} a_{\text{RD}}^4}}_{(a_f/a_{\text{RD}})^{1-3w}} \underbrace{\frac{\rho_{\text{RD}} a_{\text{RD}}^4}{\rho_o a_o^4}}_{(g_{\text{RD}}/g_o)(g_{s,o}/g_{s,\text{RD}})^{4/3}}. \quad (23)$$

The first factor follows from  $\rho_{\text{RD}} = \rho_f (a_f/a_{\text{RD}})^{3(1+w)}$ , so that  $a^4$  against  $a^{3(1+w)}$  leaves the power  $1 - 3w$ ; for time-dependent  $w$  it is

$$\epsilon_f \equiv \left( \frac{a_f}{a_{\text{RD}}} \right)^{1-3w} \rightarrow \exp \left[ - \int_{a_f}^{a_{\text{RD}}} (1 - 3w) d\ln a \right]. \quad (24)$$

The second factor uses  $\rho a^4 \propto g T^4 a^4 \propto g g_s^{-4/3}$  during the thermal phase. Multiplying by  $h^2$  (paper's Eq. (2.18), first line):

$$\boxed{h^2 \Omega_{\text{GW}}(f) = h^2 \Omega_{\text{rad}} \epsilon_f \left( \frac{g_{s,o}}{g_{s,\text{RD}}} \right)^{4/3} \left( \frac{g_{\text{RD}}}{g_o} \right) \Omega_{\text{GW}}^{(f)}(k)} \simeq 1.6 \times 10^{-5} \epsilon_f \Omega_{\text{GW}}^{(f)}(k). \quad (25)$$

$\epsilon_f < 1$  (suppression) if there is a phase with  $w < 1/3$  between the end of production and RD. Since  $\Omega_{\text{GW}}$  is defined per logarithmic interval and  $f \propto k$  at fixed time,  $d\ln f = d\ln k$ : the spectrum is mapped point by point,  $h^2 \Omega_{\text{GW}}(f(k))$ , with no Jacobian.

Note the difference between the two factors:  $\epsilon_i$  (frequency) involves the whole history from  $t_i$ , because  $k$  is defined at  $t_i$ ;  $\epsilon_f$  (amplitude) only the history after  $t_f$ , because  $\Omega_{\text{GW}}^{(f)}$  is normalized at  $t_f$ . They are related by  $\epsilon_f = \epsilon_i \exp[\int_{a_i}^{a_f} (1 - 3w) d\ln a]$ .

### 8.4 Implementation

**Lattice quantities in physical units.** With  $f_* = M_P$  and  $a_i = 1$ :

$$k [\text{GeV}] = \tilde{k} \omega_*, \quad \rho_i = \tilde{E}_{\text{tot}}(\tilde{t} = 0) M_P^2 \omega_*^2, \quad \omega_* = \sqrt{\lambda_k} M_P x_i^{(k-2)/2}, \quad (26)$$

where  $\tilde{E}_{\text{tot}}$  is read from `average_energies.txt` and  $\omega_*$  is recomputed from `run.in` exactly as in the model headers (Sec. 4). Conversion to Hz: in natural units Eq. (20) gives  $f$  in GeV, and  $f [\text{Hz}] = f [\text{GeV}]/\hbar$  with  $1 \text{ GeV}/\hbar = 1.519267 \times 10^{24} \text{ s}^{-1}$ .

**The expansion factors.** The integral  $\int (1 - 3w) d\ln a$  is split at  $t_f$ :

$$\epsilon_i = \exp \left[ - (I_f + I_{\text{post}}) \right], \quad \epsilon_f = \exp \left[ - I_{\text{post}} \right], \quad (27)$$

- $I_f = \int_{a_i}^{a_f} (1 - 3w) d\ln a$  is computed *from the lattice*. The equation of state is  $w = p/\rho$  with the volume-averaged energies of the two scalars,  $\rho = K + G + V$  and  $p = K - G/3 - V$  ( $K$  kinetic,  $G$  gradient,  $V$  potential; GWs are negligible).  $\ln a$  is interpolated from `average_scale_factor.txt` to the times of `average_energies.txt`, and the integral is done with the trapezoidal rule. Integrating the instantaneous  $w$  automatically averages over the inflaton oscillations.

- $I_{\text{post}} = (1 - 3w_{\text{post}})N_{\text{post}}$  describes the unknown history between the end of the run and the onset of RD:  $N_{\text{post}} = \ln(a_{\text{RD}}/a_{\text{f}})$   $e$ -folds at constant  $w_{\text{post}}$  (notebook variables `N_POST`, `W_POST`). The default  $N_{\text{post}} = 0$  (instant RD at  $t_{\text{f}}$ ) gives  $\epsilon_{\text{f}} = 1$ . Since  $w \simeq 0.24 < 1/3$  at the end of the runs, this is an upper bound on both  $f$  and  $h^2\Omega_{\text{GW}}$ .

**Constants.**  $T_0 = 2.7255 \text{ K}$ ,  $g_o = 3.36$ ,  $g_{s,o} = 3.91$ , so  $\rho_o = \frac{\pi^2}{30}g_o T_0^4 = 3.36 \times 10^{-15} \text{ eV}^4$ ;  $h^2\Omega_{\text{rad}} = 4.15 \times 10^{-5}$ , which is consistent with this  $\rho_o$  ( $\rho_c/h^2 = 8.10 \times 10^{-47} \text{ GeV}^4$ );  $g_{\text{RD}} = g_{s,\text{RD}} = 106.75$  (full SM). The  $g$  factors are  $(g_{s,o}/g_{s,\text{RD}})^{1/3}(g_o/g_{\text{RD}})^{-1/4} = 0.79$  in Eq. (20) and  $(g_{s,o}/g_{s,\text{RD}})^{4/3}(g_{\text{RD}}/g_o) = 0.39$  in Eq. (25).  $M_P = 2.435 \times 10^{18} \text{ GeV}$ .

**Spectra at intermediate times.** `gw.today(name, idx)` can redshift the spectrum of any output time, treating that time as  $t_{\text{f}}$ . Then  $I_{\text{f}}$ , and so the  $k \rightarrow f$  map, changes with  $t_{\text{f}}$ . The same map  $f/\tilde{k}$  is used to plot the field spectra against the frequency today.

## 8.5 Results for the two runs

Runs `runs/E_starobinsky_gw` and `runs/T_k2_a1_gw` ( $N = 64$ ,  $k_{\text{IR}} = 0.5$ ,  $q_0 \approx 10^4$ ), redshifted from  $\tilde{t}_{\text{f}} = 290$  with  $N_{\text{post}} = 0$ :

| Run                                      | $\omega_*$ [GeV]      | $\rho_i^{1/4}$ [GeV]  | $a_{\text{f}}$ | $\epsilon_i$ | $f/\tilde{k}$ [Hz] | $f_p$ [Hz]         | $h^2\Omega_{\text{GW}}(f_p)$ |
|------------------------------------------|-----------------------|-----------------------|----------------|--------------|--------------------|--------------------|------------------------------|
| E, Starobinsky, $g = 4.5 \times 10^{-3}$ | $3.35 \times 10^{13}$ | $5.84 \times 10^{15}$ | 18.9           | 0.126        | $1.57 \times 10^8$ | $2.6 \times 10^9$  | $1.6 \times 10^{-9}$         |
| T, $k = 2$ , $g = 1.7 \times 10^{-3}$    | $1.76 \times 10^{13}$ | $5.47 \times 10^{15}$ | 26.1           | 0.094        | $8.16 \times 10^7$ | $1.35 \times 10^9$ | $2.1 \times 10^{-9}$         |

$\epsilon_i \approx 0.1$  corresponds to an average  $\langle w \rangle \approx 0.1$  over the  $\sim 3$   $e$ -folds of the run, i.e. a frequency shift by  $\epsilon_i^{1/4} \approx 0.6$  relative to instant RD at  $t_i$ . A matter phase of  $N_{\text{post}} = 2$   $e$ -folds after the run lowers  $f$  by  $e^{-1/2} \approx 0.61$  and  $h^2\Omega_{\text{GW}}$  by  $e^{-2} \approx 0.14$  (dashed curves in the notebook). These numbers are not converged: the GW peak ( $\tilde{k} \simeq 17$ ) is close to the lattice cutoff  $\tilde{k}_{\text{max}} \simeq 28$ , so the runs must be repeated at  $N = 128$ – $256$  before quoting them.

## 9 How to build and run

```
# from the repository root; cmake, FFTW and a C++ compiler must be on PATH
# (e.g. activate the cosmolattice conda environment first)
mkdir build_attractorE && cd build_attractorE
cmake .. -DMODEL=attractorE && make -j8
./attractorE input=../models/parameter-files/attractorE_k2_a1_starobinsky.in
# any key can be overridden on the command line, e.g. N=128 tMax=500 g=1e-4
```

For the T-model replace `attractorE` with `attractorT`. To make a new benchmark, run the helper with the desired  $(\alpha, k, \kappa, N_*)$  and copy its output into a copy of one of the existing `.in` files.

## 10 Open points

- Fix  $N_*$  self-consistently: for  $k \geq 6$  measure the fragmentation time and temperature on the lattice and feed them back into the paper’s App. B expression for  $N_*$ , which depends on  $\max[T_{\text{RH}}, T_{\text{frag}}]$ .
- Choose the daughter coupling  $g$  (and possibly trilinear or fermionic decay channels) for the reheating study.
- For  $k \geq 6$ , after fragmentation the equation of state tends to  $w = 1/3$ ; consider  $\alpha_t = 1$  for long runs.
- Tune lattice settings (Sec. 6) with resolution tests.